

Frontend Engineer Assignment

Design System + Analytics Dashboard (React/Next.js)

Overview

This assignment has three pillars: **CSS architecture** (design system with tokens and components), **JavaScript/React mastery** (advanced patterns, performance, real-world data handling), and **problem-solving** (advanced CSS challenges and architectural decisions).

We're evaluating your ability to **architect** frontend systems — not just write code that works, but code that scales.

What We're Looking For

- CSS architecture thinking (tokens, themes, scalability)
 - Advanced React patterns (custom hooks, context, performance optimization)
 - TypeScript proficiency (proper typing, generics, discriminated unions)
 - State management and data flow design
 - Creative problem-solving with CSS and JavaScript
 - Clear technical communication
-

Tech Stack

- **Required:** Next.js 14+ (App Router) with TypeScript
 - **Styling:** CSS Modules, Sass, or Tailwind CSS (no component libraries — no MUI, Chakra, Ant Design, shadcn, etc.)
 - **State:** React built-in state + Context. You may use Zustand if you prefer — justify your choice.
 - **Data fetching:** Native fetch, React Server Components, or TanStack Query — your choice, justify it.
 - **No:** jQuery, UI component libraries, JS animation libraries (GSAP, Framer Motion)
-

Part A — Design System Tokens & Components (20%)

Design Tokens

Create a token system using **CSS custom properties** that powers your entire design system. Define tokens in a dedicated file (e.g., `tokens.css` or `_tokens.scss`).

Required Token Categories:

None

Colors:

- Primary (with 3 shades: light, base, dark)
- Secondary (with 3 shades)
- Neutral scale (50, 100, 200, ..., 900)
- Semantic: success, warning, error, info
- Surface colors (background, card, elevated)
- Text colors (primary, secondary, disabled, inverse)

Spacing Scale:

- Consistent scale (e.g., 4px base: 4, 8, 12, 16, 24, 32, 48, 64)
- Named semantically: `--space-xs`, `--space-sm`, `--space-md`, etc.

Typography:

- Font families (heading, body, mono)
- Size scale (xs through 2xl, minimum 6 sizes)
- Weight scale (regular, medium, semibold, bold)
- Line-height scale (tight, normal, relaxed)

Shadows:

- Elevation scale (sm, md, lg, xl) – used for card hover, dropdowns, modals

Breakpoints:

- sm, md, lg, xl (define the values, document them)

Optional bonus tokens: Borders & Radii (border radius scale, border widths), Transitions (duration scale, easing functions) — include these if time permits.

Theme Support

Implement **light and dark themes** using your token system.

Requirements:

- Theme switching using CSS custom properties (redefine token values under a `.theme-dark` class or `[data-theme="dark"]` attribute)
- Toggle mechanism: a button or interactive element that swaps themes (JavaScript is fine for the toggle)
- All components must look correct in both themes without any component-level theme overrides
- Smooth transition when switching themes (colors fade, not snap)

Components

Build the following 3 components using your token system. Every color, spacing, font-size, shadow, and radius value must reference a token — **no hardcoded values**.

Component 1: Button

Variant	States
Primary (filled)	Default, Hover, Active, Focus, Disabled, Loading
Secondary (outlined)	Default, Hover, Active, Focus, Disabled, Loading
Ghost (text only)	Default, Hover, Active, Focus, Disabled, Loading

Size	Specs
Small	Compact padding, smaller font
Medium	Default
Large	More padding, larger font

Details:

- Loading state: replace text with a CSS-only spinner animation (no images/SVGs)
- Focus state: visible focus ring (accessibility)
- Disabled: reduced opacity, `not-allowed` cursor
- Icon support: button should accommodate a leading or trailing icon without breaking layout

Component 2: Input Field

- **Floating label:** label sits inside the input, floats above on focus or when filled
- **States:** default, focused (with highlight border), error (red border + error message below), disabled, read-only
- **Helper text:** optional text below the input (switches to error message in error state)
- **Character counter:** optional, shows "12/100" aligned right below input

- **Prefix/suffix support:** e.g., "\$" prefix or ".com" suffix, visually attached to the input

Floating label animation must be smooth — use `transform` and `scale`, not `top`/`font-size` changes, for best performance.

Component 3: Card

- **Slots:** header area, body/content area, footer area (all optional)
- **Hover:** subtle elevation increase (shadow transition)
- **Variants:** default (with border), elevated (with shadow, no border)
- Cards should accept any content without breaking layout

Part B — Analytics Dashboard Layout (15%)

Using the components and tokens from Part A, build an analytics dashboard layout.

Layout Structure

None

```
+-----+
|               Top Navigation Bar               |
+-----+
|      |               Stats Cards Row           |
| Side |               |
| bar  |               |
|      |               |
|      | Data Table |
| (nav | (with sticky header) |
| with |               |
| icons)|               |
|      |               |
+-----+
```

Top Navigation Bar

- **Left:** Logo/app name, breadcrumb (e.g., "Dashboard > Analytics")
- **Center:** Search input (from your design system)
- **Right:** Theme toggle, user avatar with dropdown
- **Dropdown:** CSS-only dropdown on avatar hover/click showing user name, email, "Settings", "Logout"
- Sticky on scroll

Sidebar Navigation

- Width: ~260px on desktop
- **Collapsible:** toggle button shrinks sidebar to icon-only mode (~64px)
- Navigation items with icon + label
- Active state highlighting
- Smooth width transition on collapse/expand

Responsive Behavior

Breakpoint	Sidebar	Navigation
Desktop (>1024px)	Visible, collapsible	Top bar
Mobile (<1024px)	Hidden, toggled via hamburger (slides in from left with backdrop)	Top bar + hamburger trigger

Main Content Area

Stats Cards Row

- 4 stat cards in a row using your Card component
- Each card: icon, metric name, large number, percentage change with up/down arrow (green/red)
- **Responsive:** wraps to 2x2 grid on tablet, vertical stack on mobile
- Use your design tokens for all spacing and colors

Data Table

- Minimum 8 rows of dummy data, 6+ columns
- **Sticky header:** header row stays fixed when scrolling the table body
- **Horizontal scroll:** on smaller screens, table scrolls horizontally with the first column frozen (sticky)
- **Alternating row colors** using `nth-child`
- **Sortable column headers:** show up/down arrows (visual only for Part B — functional sorting in Part C)
- **Row hover:** subtle background highlight
- Pagination controls below the table (Previous, page numbers, Next) — visual only for Part B

Part C — JavaScript, React & Next.js Skills (30%)

This section tests advanced React patterns, performance thinking, and real-world JavaScript skills. Build these features into the dashboard app.

1. Data Layer & Server Components (Next.js)

Build a realistic data fetching layer for the dashboard.

Requirements:

- Create a mock API using Next.js **Route Handlers** (`app/api/`):
 - `GET /api/stats` — returns stats card data
 - `GET /api/analytics` — returns chart data with date ranges
 - `GET /api/users` — returns paginated user list for the data table (supports `?page=1&limit=10&sort=name&order=asc&search=john`)
- All responses should have realistic delays (200-800ms random) and follow a consistent response format:

None

```
type ApiResponse<T> = {
  data: T;
  meta?: { page: number; totalPages: number; totalItems: number };
  error?: string;
}
```

- Use **React Server Components** for the initial dashboard load (stats cards)
- Use **Client Components** only where interactivity is needed (table sorting, filters, search)
- Implement proper **loading.tsx** and **error.tsx** boundary files

What we evaluate: Server vs client component decisions, API design, TypeScript generics, Next.js App Router understanding.

2. Data Table — Full Interactivity

Make the data table from Part B fully functional.

Requirements:

- **Server-side pagination:** clicking Next/Previous fetches the next page from your API
- **Sorting:** clicking column headers sorts the data (server-side — sends sort params to API)
- **Search:** debounced search input (300ms) that filters data server-side
- **URL state sync:** all table state (page, sort column, sort order, search query) is reflected in URL params
 - e.g., `?page=2&sort=name&order=desc&search=john`
 - Navigating directly to that URL restores the exact table state
 - Use `useSearchParams` or Next.js `searchParams` in server components
- **Loading states:** skeleton rows while fetching, not a full page spinner
- **Empty state:** "No results found" when search/filters return nothing

- **Optimistic column sort indicators:** immediately show sort direction on click, before data returns

What we evaluate: URL-driven state, pagination UX, debounce implementation, loading state granularity.

Part D — Advanced CSS Problem-Solving Challenges (15%)

Complete **any 2 of the following 4 challenges**. These test advanced CSS knowledge and creative problem-solving.

Challenge 1: CSS-Only Multi-Step Form Wizard

Build a form with 3-4 steps that users can navigate through **without any JavaScript**.

Requirements:

- Progress bar at the top showing current step (filled segments)
- Each step is a fieldset with form inputs
- "Next" and "Previous" navigation between steps
- Current step's inputs are visible; other steps are hidden
- Smooth transition/slide animation between steps
- Final step shows a summary or success state

Hint: Use radio buttons with `:checked` and sibling/general sibling selectors to control visibility.

Evaluation: We're looking for a clean, scalable approach — not a brittle hack that barely works.

Challenge 2: Container Queries

Make the **Stats Cards** from Part B responsive based on their **container width**, not the viewport.

Requirements:

- When a card's container is wide (>300px): show icon left, text right (horizontal layout)
- When a card's container is narrow (<300px): stack icon on top, text below (vertical layout)
- Demonstrate this by showing the same card component in the sidebar (narrow) and in the main content area (wide) — same HTML, different layouts based on container
- Use `@container` queries with proper `container-type` setup

Why this matters: Container queries are a fundamental shift in responsive design thinking. We want to see that you understand when and how to use them.

Challenge 3: CSS-Only Notification Toast System

Build a toast notification system that works **without JavaScript**.

Requirements:

- Toasts appear in the top-right corner of the viewport
- Stack vertically (newest on top)
- Each toast auto-dismisses after a few seconds using `animation-delay` and `animation-fill-mode`
- Types: success (green), error (red), warning (yellow), info (blue)
- Each toast has: icon, message text, close button
- Entry animation: slide in from right
- Exit animation: fade out and slide up
- Trigger mechanism: use `:target` pseudo-class with anchor links (e.g., clicking a button navigates to `#toast-1` which triggers its appearance)
- Support at least 3 toasts visible simultaneously

Hint: Combine `:target`, `@keyframes`, `animation-delay`, and `animation-fill-mode: forwards` to control the lifecycle.

Challenge 4: Fluid Typography & Spacing System

Implement a typography system that scales **smoothly and continuously** from mobile to 4K displays without using breakpoints.

Requirements:

- Use `clamp()` for all font sizes, with calculated min, preferred, and max values
- Document your formula/approach (e.g., based on viewport width, what's your scaling factor?)
- Apply to:
 - All 6+ type sizes in your token system
 - Spacing tokens (margins, paddings also scale fluidly)
 - Component internal spacing
- Create a demo page showing:
 - A type specimen (all heading levels + body text + small text)
 - The same layout at 320px, 768px, 1440px, and 2560px (screenshots or responsive demo)
- No text should ever be too small to read (<14px effective) or absurdly large

Evaluation: We're looking for a principled approach, not random `clamp()` values. Bonus if you explain the math behind your min/max/preferred calculations.

Part E — Written Reflection (5%)

In your README or a separate `NOTES.md`, answer these questions briefly (2-4 sentences each):

1. **Server vs Client components:** How did you decide which components should be Server Components vs Client Components? What was the trickiest boundary?
 2. **Trade-offs:** What corners did you cut due to time constraints? What would you improve with another 4 hours?
 3. **Hardest part:** What was the most challenging aspect of this assignment, and how did you work through it?
-

Submission Instructions

1. **Push your code** to a public GitHub repository
 2. **Deploy** to Vercel (recommended for Next.js) or any free hosting
 3. **Repository must include:**
 - README.md with:
 - Live demo link
 - How to run locally
 - File structure overview
 - Which Part D challenges you completed (and why you chose those 2)
 - NOTES.md with Part E answers
 - Clean git history showing your process (meaningful commits, not one giant commit)
 4. **Send us:**
 - GitHub repo link
 - Live demo link
-

Evaluation Rubric

Criteria	Weight	What We Look For
JavaScript, React & Next.js (Part C)	30%	Hooks, data fetching, server/client split, URL-driven state, performance
CSS architecture & token system (Part A)	20%	Principled tokens, proper theming, no hardcoded values
Dashboard layout & responsiveness (Part B)	15%	Works across breakpoints, sidebar behavior, table handling
CSS problem-solving (Part D)	15%	Creative, working solutions demonstrating advanced CSS

Code organization & architecture	10%	File structure, separation of concerns, clean git history
Component quality & reusability	5%	Components work in isolation, proper TypeScript types, all states
Written reflection (Part E)	5%	Clear thinking, trade-off awareness, mature engineering mindset

What Sets Great Submissions Apart

- **Server/Client boundary decisions:** clear reasoning about what runs where
 - **URL-driven state:** the entire dashboard state is shareable via URL
 - **Token consistency:** every single value traces back to a token — zero magic numbers
 - **TypeScript rigor:** proper generics, discriminated unions, no `any` escape hatches
 - **Performance awareness:** optimizations that matter, not premature optimization everywhere
 - **Part D creativity:** solving CSS problems in ways that are clean and maintainable, not just clever hacks
 - **Part E honesty:** acknowledging trade-offs and showing mature engineering thinking
-

Tips

- **Spend time planning your architecture** before writing code — token system, folder structure, data flow.
 - **Commit frequently** with descriptive messages — we review git history.
 - **Don't gold-plate** — a well-architected 80% solution beats a messy 100% solution.
 - **Server Components by default** — only use Client Components when you need interactivity or browser APIs.
 - **Part D:** Pick the 2 challenges that best showcase your strengths. Quality over quantity.
-

Questions?

If you have any questions about the requirements, email us at [INSERT EMAIL]. Asking thoughtful questions is a positive signal — it shows you think before you build.

Good luck! We're looking forward to seeing your approach.